

Frontend Documentation: AI-Based Web Exam Simulation

1. Overview

The frontend is a web-based AI system that automates the core workflows for the application. The solution must be fully web-based, meaning there is no software installation required for the end-users. It acts as the interactive layer connecting professors, students, and the AI models.

2. Technology Stack & Configuration

Based on the provided configuration files and system architecture:

- **Core Framework:** The frontend user interface is built with React.
- **Dependency Management:** Project dependencies and scripts are managed via npm, as indicated by the presence of package.json and package-lock.json.
- **Custom Build Configuration:** The config-overrides.js file signifies that the project overrides the default Webpack configuration (likely using tools like react-app-rewired or craco). This allows developers to add custom loaders or polyfills without needing to "eject" the standard React build environment.
- **Deployment:** The frontend is containerized using Docker. The .dockerignore file ensures that local development artifacts (such as node_modules or local secrets) are excluded from the image build process to keep it lightweight and secure.

3. Core Workflows & Interfaces

The frontend application is structured to handle two primary distinct workflows:

A. Professor Workflow (Exam Generation & Management)

- **Material Upload:** Professors have a dedicated interface to upload lecture materials (PDFs, text). This interacts with the backend's PDF upload endpoints.
- **Exam Review & Editing:** Once the system processes the uploaded content using NLP and RAG, the frontend provides a dashboard where professors review, edit, and finalize the generated exams.
- **Dashboarding:** The UI allows staff to view their generated exams, track participant metrics, and manage the student enrollment lists via the backend endpoints.

B. Student Workflow (Exam Execution)

- **Identity Verification Room:** Before accessing the exam, students authenticate via facial recognition and ID card OCR. The frontend utilizes browser APIs to access the webcam, captures a live image alongside an uploaded ID photo, and sends them to the backend verification engine.
- **Exam Execution Environment:** The testing interface displays the questions and collects the student's answers. The session is monitored (e.g., fullscreen mode, copy/paste blocking) to maintain academic integrity. Once completed, student responses are collected and evaluated automatically.
- **Fallback Systems:** If false identification or OCR errors occur, the UI provides a flow for the student to trigger a manual check by a provider.